

Agent Traffic Flow Management: forecasting and shaping LLM demand from in-flight agent sessions on a shared GPU pool

Alexander Aghili. Build 3fbd02b + feature branch of 2026-09-27; 151 tests; simulator and forecast results as of this date. Prepared with Claude Code.

Abstract. Fleets of LLM agents alternate between short model calls and tool executions that last from milliseconds to hours. On a shared serving pool this produces demand that is bursty, heavy-tailed and, we argue, largely *predictable*: most of what arrives in the next few minutes is already in flight, running a tool whose remaining time can be estimated. ATFM (Agent Traffic Flow Management) treats the pool the way air-traffic flow management treats airspace: a demand board forecasts KV-cache and prefill demand per horizon from the live state of every session; a proxy admits, orders and, for deferrable work, holds calls against that forecast; and every hold is charged its measured cost. This report describes the design (twelve recorded decisions), the implementation (a forecast core, laptop plumbing on NVIDIA Dynamo, and a closed-loop fleet simulator; 4.6 kLOC of Python, 151 tests), and the evidence to date. On two public corpora of real agent sessions (TraceLab, 4,265 sessions; AgentX, 393 Claude Code sessions with sub-agent fan-out), conditioning on in-flight state beats a Kalman history baseline by 3 to 5x on q90 pinball loss at 5 to 15 minute horizons and is never worse at 10 s (H1a). On 202 observation points inside real long tools (cmake builds, data pipelines, the numpy test suite), live progress cuts remaining-time error 2.8x over elapsed time alone (H1b). In the simulator, under load, forecast-driven holds raise interactive SLO attainment by 9 points over engine-native priority at a 24% background completion-time cost, a no-hold ablation shows the holds are what buy the tail, and a ThunderAgent-style pause baseline reaches the same SLO at 2.6x the background cost (H2, synthetic, ordering only). We state what remains unvalidated, chiefly the KV model, and give the design of the two-GPU study that would validate it.

1. Problem and claim

An agent session is a loop: the harness sends a prompt, the model streams a reply, the reply launches a tool (a shell command, a test suite, a build, a query, a sub-agent), the tool's output becomes the next prompt. Between model calls the session holds no GPU compute but its context, and therefore its KV cache, is still valuable: the next call reuses it as prefix. When many sessions share a pool, the serving system sees a stream of calls whose timing is set elsewhere, by tools running on CPUs, CI runners and remote services.

Today's serving stacks treat each call as independent. Priority fields, prefix-aware routing and KV eviction policies react to what has arrived. Yet the sessions that will arrive in the next thirty seconds to fifteen minutes are, for the most part, already known: they are the ones whose tool started a while ago. Their return time is uncertain but not unknowable, and their context size is known exactly. The central claim of this program is that this in-flight information is worth acting on, and the three hypotheses below make it falsifiable.

- **H1a (demand board).** In-flight session state (phase, elapsed time, context size) forecasts fleet KV and prefill demand at 30 s to 15 min better than history-based predictors of the same series.
- **H1b (sidecar).** Live tool progress improves the forecast beyond elapsed time, measurable only on long-tool workloads.
- **H2 (controller).** A proxy that admits, orders and holds calls against the forecast improves the interactive tail at a stated and measured cost to deferrable work, in closed loop.

The claims are deliberately separable. A demand board can be right and useless if the controller cannot act on it; a controller can help by rules alone with no forecast. The evaluation therefore scores forecasts on their own terms (proper scoring rules against realized demand) and policies only in closed loop, where agents react to the reply times they receive.

Production evidence for the workload shape. DeepSeek's DSec report (arXiv 2609.22978, September 2026) measures the CPU side of this workload at scale: about 380,000 concurrent sandboxes and 3 million a day per unit; median sandbox lifetime 15 to 17 minutes with a p99 over three hours; about 90% of sandboxes use at most 5% of their requested CPU on average, the tool phase being "short CPU bursts separated by periods" of waiting on the model; and from DeepSeek-V4.1 the agent loop runs off the GPU pods and calls model serving as a separate service. That is exactly the regime this design assumes: long-lived, mostly idle sessions with sparse, bursty GPU demand, decoupled from the pool. The report says nothing about inference serving or forecasting; it is evidence for the demand's shape, not a method.

2. Related systems

Four lines of work touch this problem. **Agent-aware serving**

(ThunderAgent-style pause and resume, Continuum-style KV time-to-live from tool start) manages the working set reactively: when memory is tight, pause or drop the session whose KV is least likely to be needed soon, judged from history or a fixed TTL. These are our baselines B2 and the working-set arm. **Demand-aware scheduling** (ConServe-style co-serving of latency-

sensitive and best-effort work) exploits the same two-class split we use, one layer down. **Serving frameworks** (NVIDIA Dynamo with its KV router, planner and `nvext.agent_hints`; vLLM and SGLang priority scheduling) provide the actuation points: a priority field the engine honours and metrics the proxy can scrape. **Sandbox platforms** (DSec, above) provide the execution side and its measured statistics. What none of them do is forecast the fleet's demand from the sessions in flight and shape admission against that forecast. That is the gap ATFM occupies, and it is why the evaluation compares against both a reactive working-set baseline and an oracle.

3. Design

The design is recorded as twelve decisions (D1 to D12) in the specification, each with the alternative rejected and the reason. The ones that shape everything below: the class-aware queue lives in a proxy above Dynamo with a *global* admission window and a stable priority *tier* written into the request hints (D1); policy claims come only from closed-loop runs, never from replayed traces (D12); forecasts are Monte Carlo samples per horizon, not moments (D5); every predictor implements one interface so that gains are attributable to information used rather than code (D6); every controller fails open (D10); and KV tiering is future work because no shipped API pins or demotes one paused session's KV today (D3).

3.1 System context

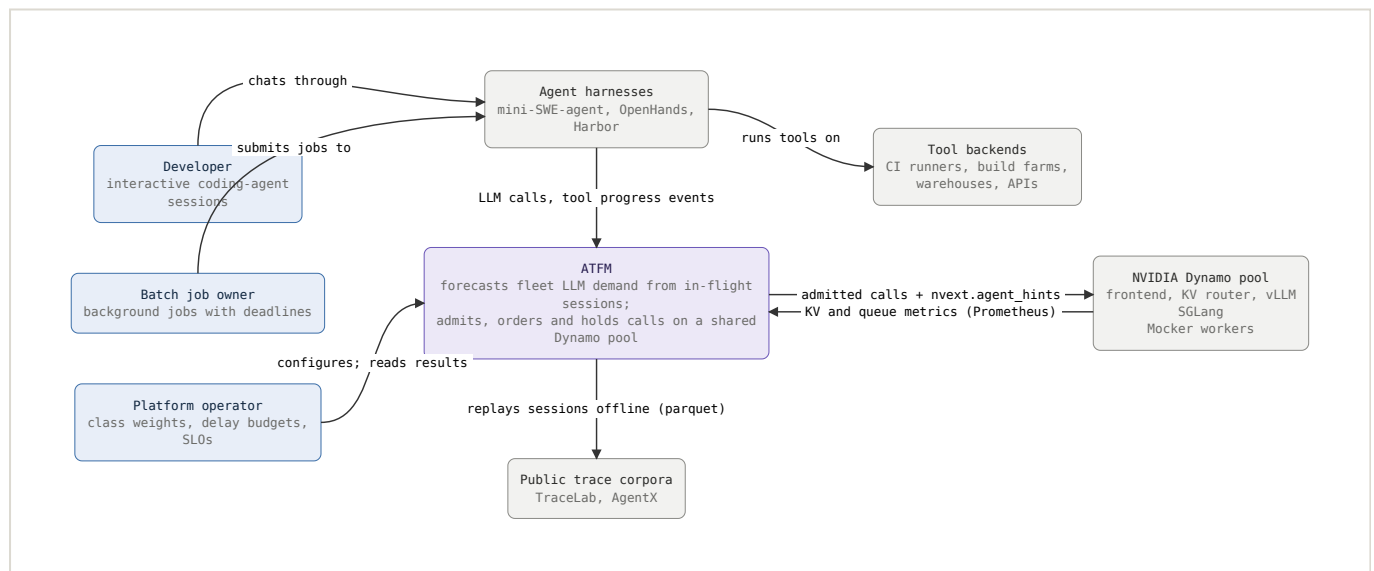


Figure 1. System context. Two paths leave every harness: the request path (LLM calls through the proxy into Dynamo) and the state path (tool progress from the sidecar). Three kinds of people depend on the system: developers driving interactive agents, owners of background jobs with deadlines, and the platform operator who sets the trade-offs. Public trace corpora feed the offline evaluation.

3.2 Containers

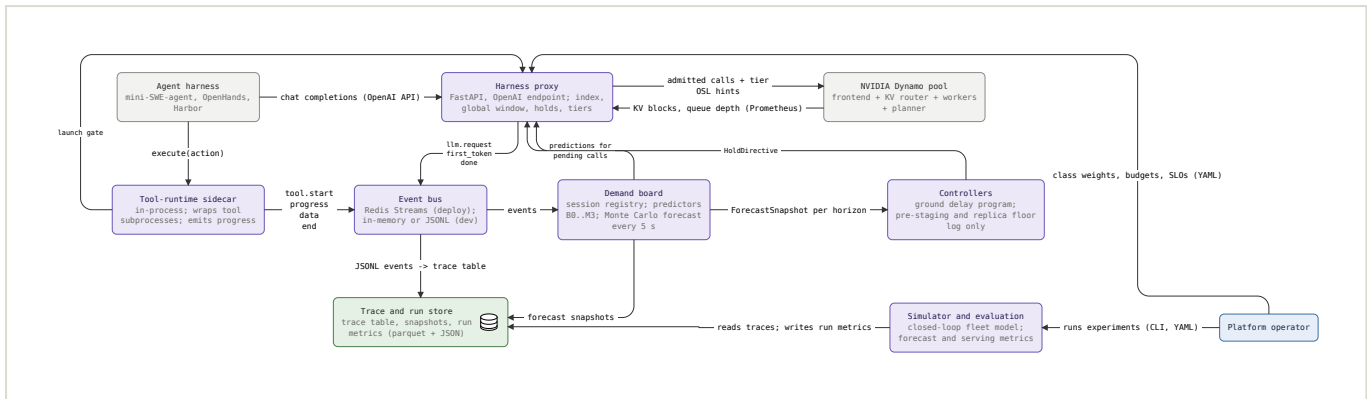


Figure 2. Containers (v1.1). Row one is the request path. Row two is the event and forecast cycle: sidecar and proxy publish to the bus, the demand board consumes it and forecasts every 5 s, the controllers turn a snapshot into hold directives that return to the proxy. Nothing on the state path can block the request path. In development the JSONL bus *is* the trace source: the same events become the canonical trace table.

3.3 Data model

Everything, real or simulated, is expressed in one event schema and one flattened trace record (D4). Events are pydantic models discriminated by `kind`: `session.start/end`, `llm.request/first_token/done`, `tool.start/progress/data/end`, `spawn.request`, `worker.metrics`. The trace record has one row per LLM call together with the tool phase that call launched: timestamps for request, first token, last token, tool start and tool end; input and output lengths; tool name, backend and command signature; the progress events (`completed/total` with phase) and data events (named metrics such as an early-stop probability) observed during the tool; and parent session for fan-out. Adapters exist for TraceLab, AgentX and the sidecar's own events. A forecast snapshot holds, per target (KV blocks, prefill tokens), per class and per horizon, an array of Monte Carlo samples plus the fraction of demand that is endogenous (from sessions already in flight).

3.4 What one turn moves through

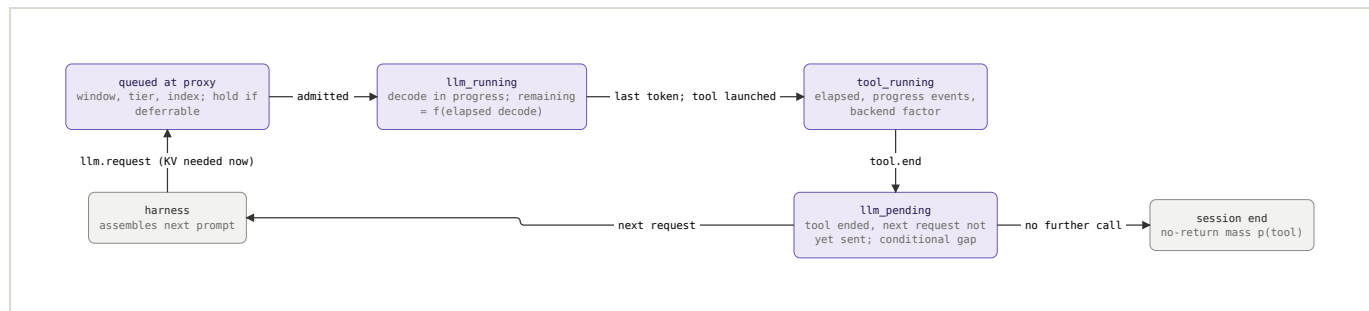


Figure 3. Session turn lifecycle and what the board conditions on in each phase. `tool_running` is where the information is: elapsed time, progress events and the backend's shared slowdown factor. `llm_pending` (tool ended, next request not yet sent) is modelled as a conditional gap, because the tool-end to next-request delay has a heavy tail in real traces (TraceLab p99 45 s, maximum 128 h). A session that never calls again is a no-return mass per tool.

3.5 The queue, the index and the tiers

Three queues exist: the proxy's hold queue (the only place a call can be delayed on purpose), the engine's own scheduler queue, and the router's choice among workers. The proxy keeps a global window of in-flight calls; when the window is full, the next call released is the one with the highest (tier, index). The tier is stable and coarse: 2 for interactive calls under deadline slack, 1 for interactive, 0 for background. It is written into `nnext.agent_hints` so that the router and engine keep class order for everything released. The index within a tier is a weighted score over expected service time, expected next-tool duration (a session that will vanish into a long tool is a good one to serve first) and context size, with a weight *beta* the operator sets. Because the window equals the batch size in the experiments so far, all queueing happens at the proxy (queue share 1.0 for proxy arms, 0 for native), which is intended: the proxy is the only place holds can be applied.

3.6 Demand board and the predictor ladder

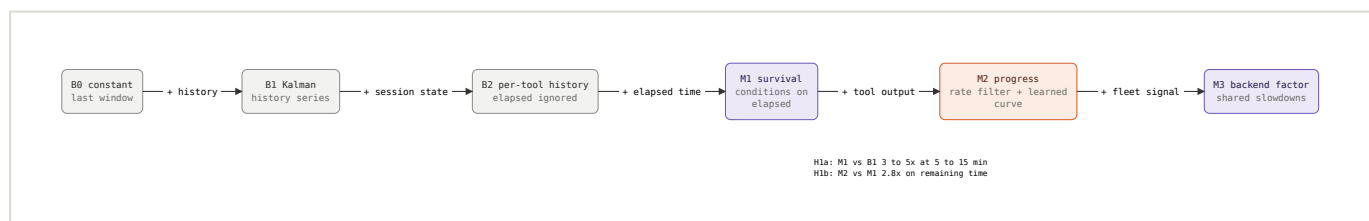


Figure 4. The predictor ladder. Each rung adds one source of information. B0 and B1 are series predictors on the realized demand history alone. B2 uses per-tool duration history but ignores elapsed time (a Continuum-style time-to-live from tool start). M1 conditions the same distribution on elapsed time (survival). M2 adds a Bayesian rate filter over live progress events and a learned per-tool progress curve mapping reported progress to time fraction (cmake percent markers and test counts are rarely linear in time). M3 adds a shared per-backend slowdown factor. Every rung returns a distribution over time-to-next-call and next-call size from the same session-state input.

The fleet forecaster draws, for each session and each sample, a return time and a next-call size, adds one level of fan-out (children a tool may spawn) and an exogenous non-homogeneous Poisson arrival process for sessions not yet started, and sums into demand per horizon. Demand truth counts each session once per horizon: the KV required on its first resumption, not every hop. Dispersion calibration fits a per-class, per-horizon variance inflation out of sample within the training data on an overlaid holdout fleet.

3.7 Ground delay: holding at a stated cost

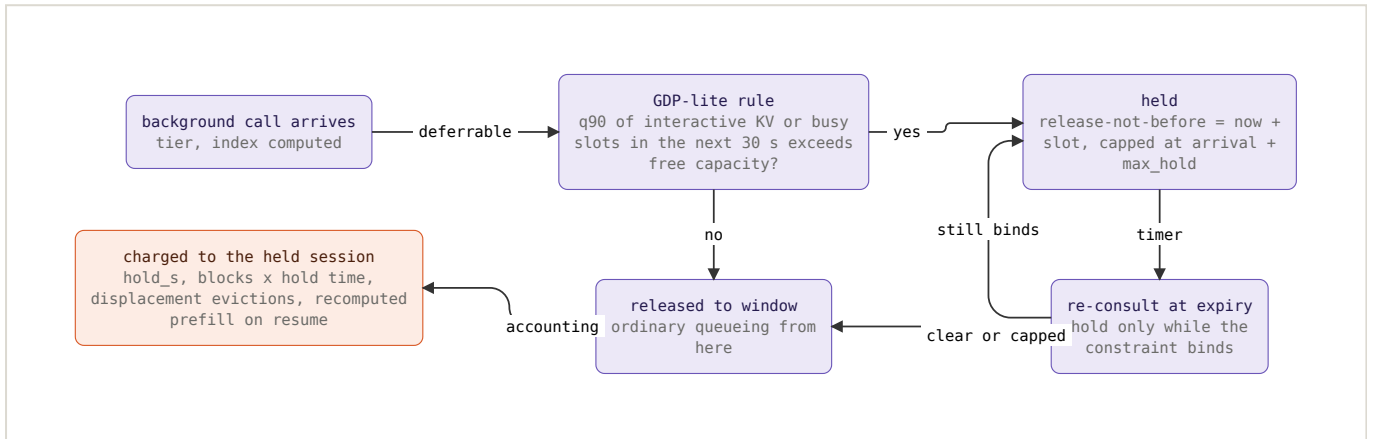


Figure 5. Hold semantics and cost accounting. The GDP-lite rule (from air-traffic ground delay programs) holds a deferrable call while the 90th percentile of forecast interactive demand over the next slot exceeds free KV capacity or free batch slots, where free capacity is what running requests do not hold, not merely what the LRU has not filled, and the slot test compares expected busy slots (calls $\times$ E[service] / slot) with free slots. A hold is capped in the simulator core at `max_hold_s` after arrival and re-consulted at expiry, so it lasts only while the constraint binds. Cost columns are measured, never assumed: seconds held, held session's resident blocks $\times$ hold time until eviction, displacement evictions attributed to held sessions, and prefix recomputed on resumption.

3.8 Sidecar

The sidecar wraps every tool subprocess in the harness. The result path is untouched and byte-exact (verified with invalid UTF-8 and a killed timeout); a separate state path parses the tool's output for progress (pytest counters, cmake and make percent markers, generic `k/N` counters, line rate) and publishes events off the request path. It classifies the command into a coarse tool class and, since this build, records a *command signature* (sizes normalised, attached flag digits kept, so `--rows 600` and `--rows 900` share a curve while `-j1` and `-j2` do not), which keys progress curves finer than the tool name. A launch gate lets a deferrable tool or spawn ask whether it may start. The sidecar is fail-open: a parser exception drops the line, a bus failure is ignored.

3.9 Simulator

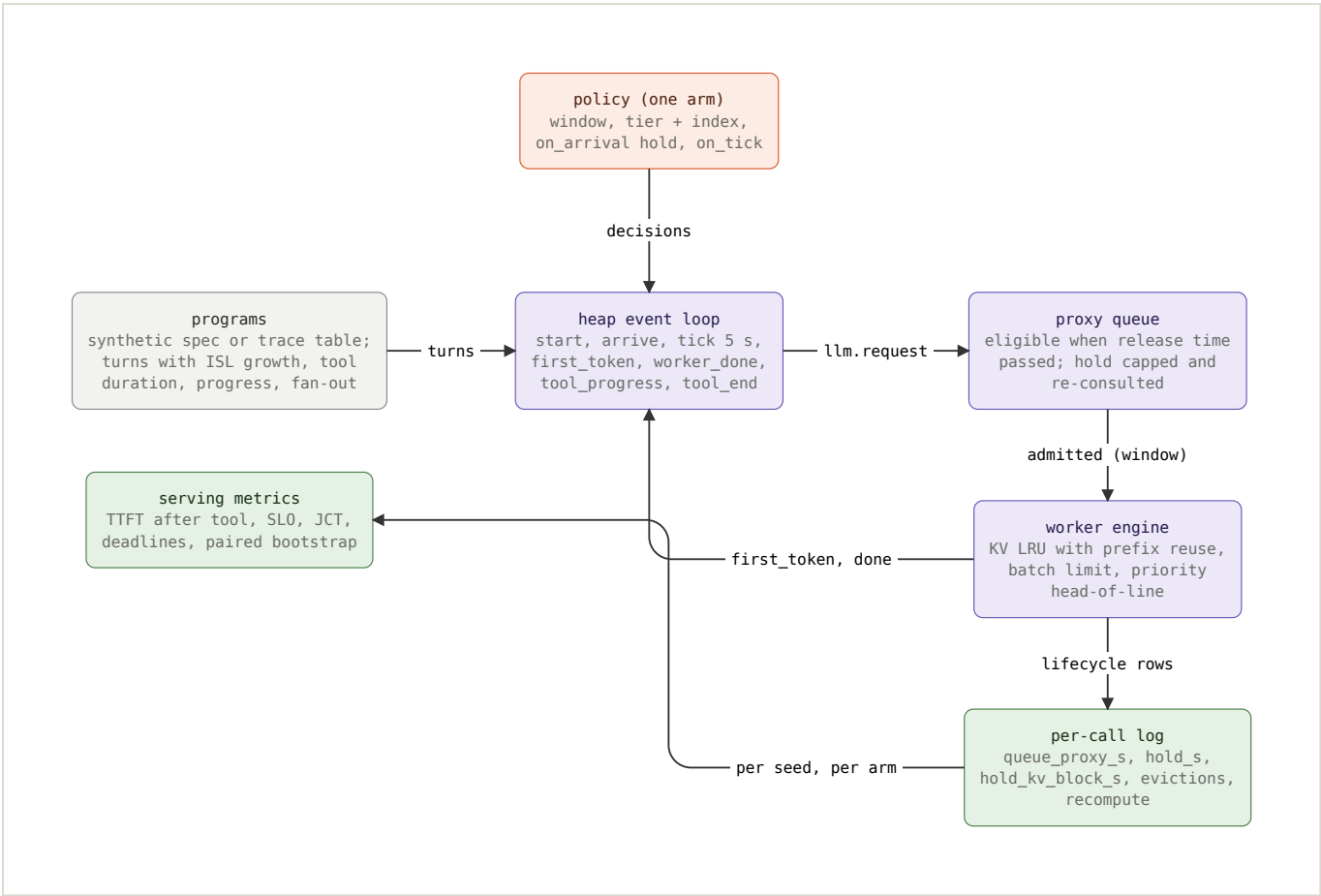


Figure 6. The closed-loop simulator. Programs (synthetic from a workload spec, or built from a trace table) are sequences of turns with prompt growth, tool duration, progress reports and fan-out. A heap event loop drives a proxy queue and KV-aware workers (LRU with prefix reuse, batch limit, head-of-line priority). Every arm is one policy object exposing window, tier and index, an arrival-time hold decision and a per-tick update. Every call leaves a lifecycle row with where it waited and what any hold cost; serving metrics are paired across arms by seed with a bootstrap over sessions.

4. Implementation

Python 3.12, uv, pydantic v2, numpy and pandas, FastAPI for the proxy and board, pytest; no Rust (D7). 55 source modules, about 4,600 lines, 151 tests (2 gated on a running Dynamo). The table lists what exists and its verification state.

Table 1. Modules and their status.

AREA	MODULES	WHAT IT DOES	STATUS
Schema	schema/trace, forecast, events	Trace row and table (nested columns as JSON in parquet), forecast snapshot, event models	done
Traces	traces/tracelab, agentx, sidecar, transform, synthetic	Corpus adapters, family-aware split and Poisson overlay, event-to-trace adapter, synthetic fleet generator	done
Board	board/state, live, replay, forecaster, calibrate, predictors/*	Session registry, replayer, Monte Carlo fleet forecaster with exogenous arrivals, dispersion calibration, ladder B0 to M3	done
Evaluation	eval/forecast, coverage, resumption, serving	CRPS, pinball, coverage, surge lead time; signal coverage; per-session remaining-time scoring; serving metrics and paired bootstrap	done
Sidecar	sidecar/core, parsers, gate, minisweagent	Byte-exact tool wrapper, parser chain, launch gate, mini-SWE-agent local and Docker environments	done
Proxy	proxy/config, index, queue, app	OpenAI-compatible endpoint, global window, tiers and index, hold directives with cap, call log, gate	done
Dynamo	dynamo/local	Mocker workers plus frontend with file discovery; starts in about 4 s on a laptop	verified
Collection	collect/jobs, driver	Scripted Docker jobs (pytest suites, cmake builds, pipelines) through the sidecar	done
Simulator	sim/programs, engine, core, policies, forecast_arm	Programs, KV engine, event loop, six arms + ablation + same-rule oracle	reviewed, fixed
Experiments	experiments/h1, h2sim; scripts/*	H1 runner and sweeps, H2 regime runner, H1b leave-one-family-out scorer	done
Controllers	pre-staging, replica floor	Logged only in v1 (D3)	future

The proxy was verified end to end against Dynamo v1.5 Mocker workers on this machine: three concurrent scripted sessions kept the window at two and gave the deadline-bound interactive session the top tier. The simulator was built test-first from a written plan, then reviewed by a fresh reviewer, which found three critical and seven important defects (a hold rule that measured LRU emptiness instead of capacity; a working-set arm whose working set included the paused sessions' own KV; hold cost conflated with ordinary window queueing; one-shot holds so the cap never engaged;

ended sessions kept in the registry; throughput computed from the specified duration rather than the realized makespan; head-of-line skipping under priority; shared child objects across overlay copies). All were fixed with pinned tests before any result below was produced.

5. Evaluation method

5.1 Corpora and collections

Table 2. Data used.

SOURCE	CONTENT	SIZE	USED FOR
TraceLab	Real Claude Code and Codex sessions with per-round timestamps and context sizes	4,265 sessions, 357k rounds	H1a; weekly time-block split, held-out weeks overlaid as a Poisson fleet
AgentX (SemiAnalysis)	Claude Code sessions with real sub-agent groups; timestamps relative to trace start; inter-request gap is unlabeled tool-or-human time	393 sessions, 1,697 sub-agent groups	H1a second corpus; family split (root plus children) and family-aware overlay
Sidecar collections	Real tools in Docker: cmake builds of fmt at -j1 and -j2, data pipelines with linear or staged progress at four sizes, numpy's full test suite (49,696 tests)	18 + 2 sessions, 18 long phases, 51k progress events per numpy run	H1b per-session scoring
Synthetic programs	Workload specs: arrival rates, turn counts, prompt growth, lognormal tool durations with signal class, fan-out, think times	3,600 s per seed, 3 seeds per regime	H2 closed loop

5.2 Scoring

Forecasts are scored per tick against realized demand with proper scoring rules: CRPS from samples, pinball loss at q90 and q95 (under-forecasts weighted 9x at q90, since a surge missed costs more than a surge over-called), 80 and 90% interval coverage, and surge lead time against the capacity line. History baselines see only realized windows, never the future. Per-session resumption scoring (H1b) places an observation every 15 s inside every tool phase longer than 30 s and scores each predictor's distribution of the remaining time against the truth, leaving one job family out at a time. Serving metrics (H2) are TTFT after a tool returns, session-weighted SLO attainment at a 2 s bound, background job completion time, deadline hit rate, recomputed prefill tokens, and the hold cost columns of

Figure 5; arms are paired by seed and differences carry 95% bootstrap intervals over sessions.

6. Results

6.1 H1a: in-flight state versus history, two corpora

Table 3. TraceLab, seed 0, 200 sessions/h. Mean q90 pinball loss of forecast KV blocks demanded within the horizon (lower is better). Predictors fit on held-in weeks; held-out weeks replayed as a fleet for 4 h.

MODEL	10 S	30 S	2 MIN	5 MIN	15 MIN
B0 constant	10409	10845	13861	16485	43105
B1 Kalman	4588	6615	10597	14186	26101
B2 per-tool history, elapsed ignored	40651	38324	33861	30628	26183
M1 survival on elapsed time	5567	5254	5679	6604	7700
M2 progress (no progress events in this corpus)	5538	5251	5654	6611	7671

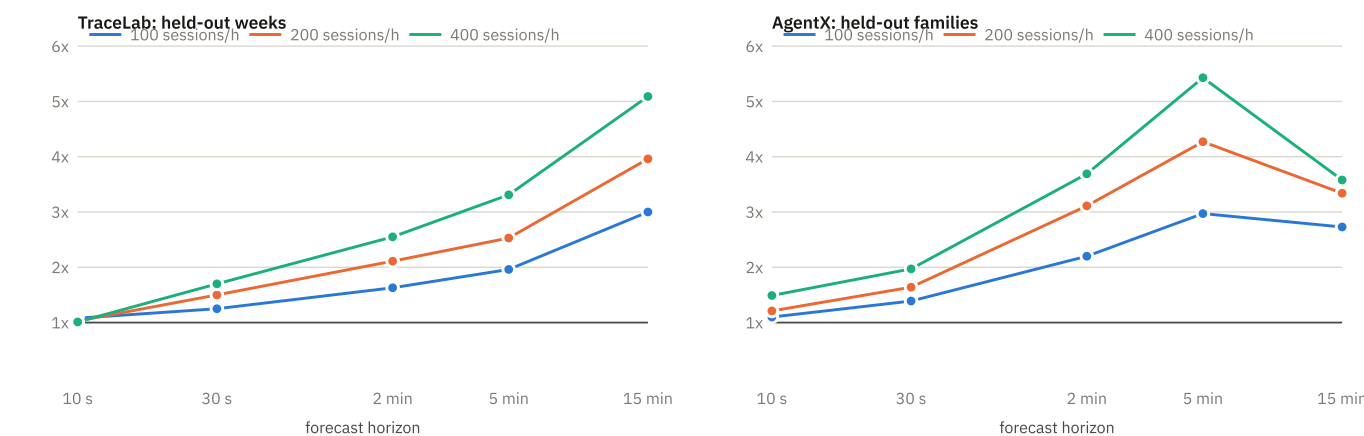


Figure 7. H1a sweep, three seeds x three arrival rates. Ratio of the Kalman baseline's q90 pinball loss to the elapsed-time model's (above 1 favours in-flight state). The advantage grows with horizon and fleet density on both corpora and is never below 1. AgentX 15-minute cells carry 95% CI half-widths of 50 to 60% of the mean (a few hundred held-out sessions dominated by long sub-agent sessions) and should be read as "about 3x"; all other cells are within 10 to 25%.

Demand at these horizons is 81 to 99% endogenous: it comes from sessions already in flight. The session models are under-dispersed (40 to 73% coverage of a 90% interval on TraceLab); out-of-sample calibration lifts that to 65 to 86% but not to 90, which says the remaining miss is bias on sessions and tools unseen in training rather than variance. B2 is the worst predictor on both corpora by a wide margin: ignoring elapsed time inside a heavy-tailed tool is the wrong thing to do, which is the quantitative case against fixed TTLs from tool start.

6.2 H1b: live progress inside real long tools

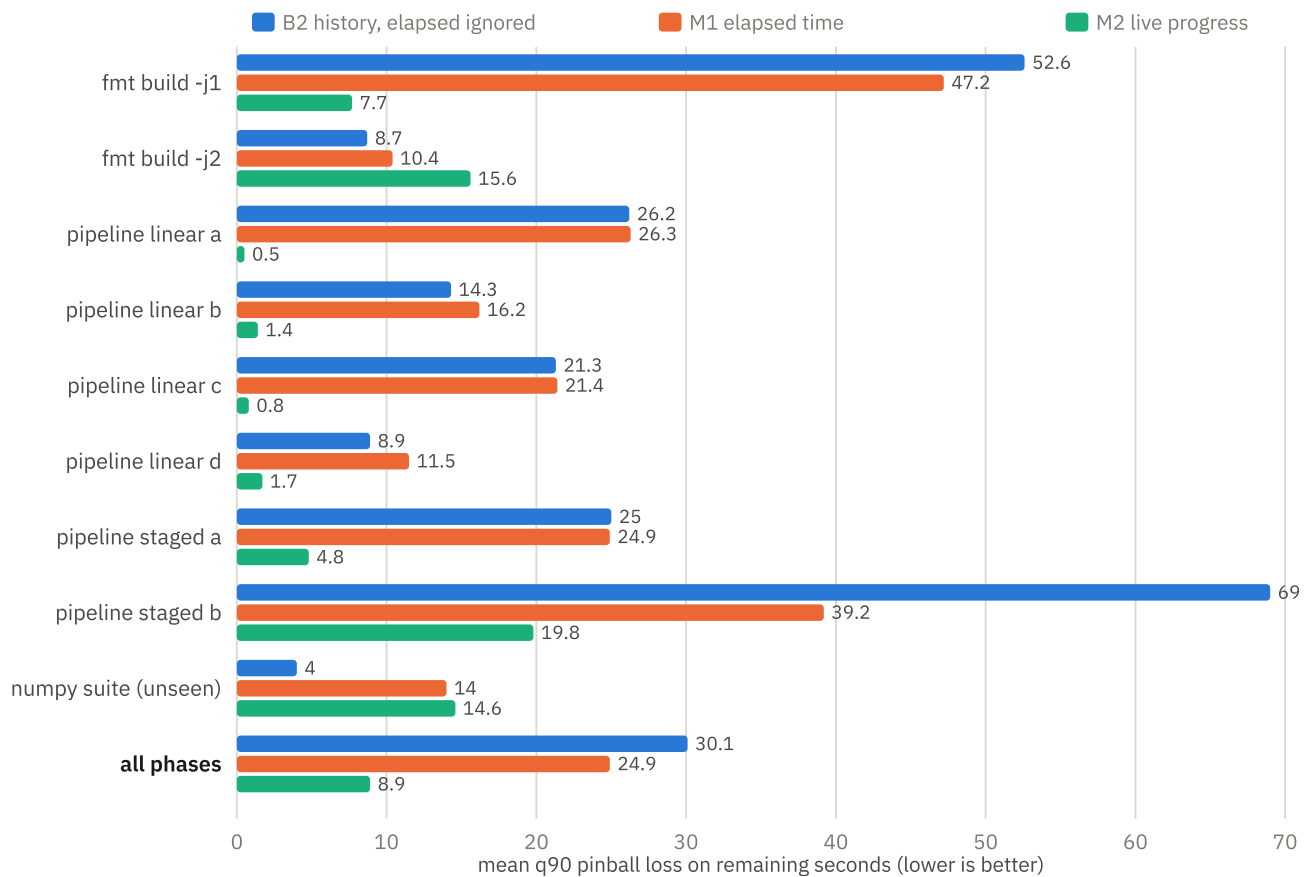


Figure 8. H1b, leave-one-job-family-out, 202 observation points per model. Mean q90 pinball loss on remaining seconds, per job family, for the three session predictors. With a linear signal (pipelines) M2 is near exact; with stage markers only it halves M1's error; on cmake builds it depends on whether a curve for that build's shape is in training. The numpy suite is the S-shaped case: 12% of tests done at a quarter of the wall time, 81% at three quarters. With the suite never seen, linear extrapolation buys nothing (14.6 vs 14.0); with one prior run in training, CRPS drops 2.9x against M1 (18.8 vs 53.7).

CRPS over all phases: B2 67.3, M1 59.0, M2 30.1. B2's small numbers on numpy and on the -j2 build are artifacts of near-identical durations across repeats and of a pooled draw that happens to sit above the truth; the TraceLab and AgentX fleets show what B2 does when durations vary. The gate for H1b was set in advance as "M2 beats M1 on a 45-minute concurrency-1 long-tool collection", and it passed on the varied-duration collection with q90 pinball 10.0 vs 28.1 before numpy was added.

6.3 H2 in the simulator: six arms, three regimes

Read this section as ordering only. The workload is synthetic and the engine's KV model (block LRU with prefix reuse, fixed prefill and decode rates) is unvalidated against a real engine. No absolute number here transfers to an H100. The simulator exists to say where each arm queues work and what a hold costs, so the GPU study measures the right things.

One engine of 8,000 KV blocks and batch 8 (6,000 and 6 in the loaded regime), Dynamo-style priority honoured with head-of-line order kept. Interactive sessions: 6 turns, 20 s think time, bash tools of a few seconds. Background: 10 turns, deadline 1,800 s, tools bash 3 s / pytest 180 s with a strong signal / build 400 s with a weak signal (*long_tool*), or bash 2 s / pytest 20 s (*short_tool*). Arms: **native** (engine priority only), **proxy_rules** (window, tiers, index, no holds), **forecast_M1** and **forecast_M2** (rules plus the GDP-lite hold on the board's forecast), **forecast_M2_nohold** (forecast only in the index), **oracle** (reads the event heap: true arrivals in the next slot, occupancy rule) and **working_set** (ThunderAgent-style: pause background programs and offload their KV when the working set exceeds budget; hysteresis release). Hold cap 600 s in these runs.

Table 4. Loaded long-tool regime (360 interactive and 600 background sessions/h), 3 seeds, paired against bootstrap intervals in brackets.

ARM	SLO SESSIONS	DIFF VS NATIVE	TTFT AFTER TOOL P95 (S)	BACKGROUND JCT DIFF (S)	DEADLINE HIT	RECOMPUTED PREFILL TOKENS	F BLI
native	0.851	—	2.68	—	0.190	4.79e7	
proxy_rules	0.867	+0.014 [−0.002, +0.028]	3.01	−35 [−90, +17]	0.247	3.82e7	
forecast_M1	0.941	+0.088 [+0.074, +0.101]	2.16	+790 [736, 842]	0.119	4.33e7	4
forecast_M2	0.946	+0.093 [+0.080, +0.105]	2.12	+789 [739, 837]	0.122	4.31e7	4
forecast_M2_nohold	0.841	−0.013 [−0.028, +0.001]	3.23	−35 [−97, +10]	0.243	4.14e7	
oracle (heap)	0.923	+0.070 [+0.054, +0.083]	2.51	+1141 [1090, 1190]	0.091	4.10e7	2
working_set	0.929	+0.076 [+0.063, +0.089]	2.29	+2082 [2035, 2136]	0.114	4.66e7	

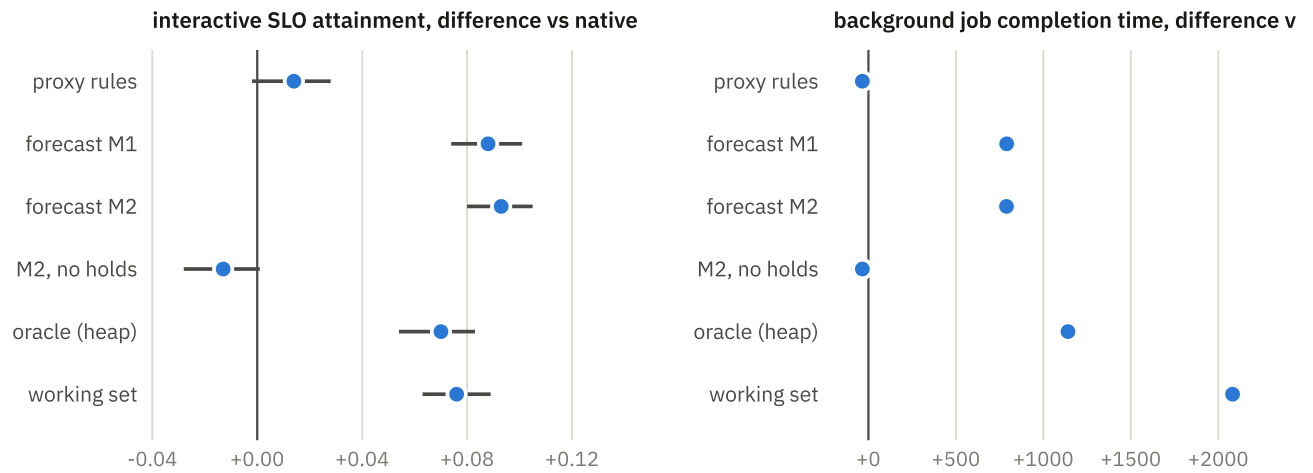


Figure 9. The trade in the loaded regime. Left: interactive SLO gain over native. Right: what background work pays for it. The forecast arms and the working-set baseline reach similar SLO; the working set costs 2.6x as much background time and 58% more recomputed prefill in the long-tool regime, because pause and resume cycles evict and recompute whole contexts. Rules alone and the no-hold ablation are within noise of native.

Three readings are safe. First, holds are what buy the interactive tail: the ablation identical to forecast_M2 except that it never holds is indistinguishable from plain rules. Second, holds are not free: 24% more background completion time, 7 points fewer deadlines, 4.2 million held KV block-seconds and about 1,200 holds per seed that ran to the 600 s cap; held calls then still wait at the window (median proxy queue 579 s for held calls, 1 s for unheld background). Third, demand-aware holds reach the same SLO as occupancy-triggered pausing at 40% of its background cost. In the two base regimes (120 and 240 sessions/h) every arm sits at 0.99 to 1.00 SLO and only the p95 TTFT after a tool moves (22% lower with holds in long_tool, 8x lower in short_tool), so those regimes cannot rank arms on the SLO.

Two things the simulator did *not* show. M1 and M2 do not separate: their TTFT, completion time and hold costs are within noise, because interactive sessions in these programs run only short tools, so progress prediction never enters the hold rule. And the heap-reading oracle is not an upper bound on the forecast arms: it uses a different rule (occupancy of the next slot from true arrivals), holds less and gets a worse tail. Both gaps are addressed by runs in flight (Section 8).

7. Limitations

- **No GPU results.** Every policy number is from the simulator; the KV model is the least validated component. The Dynamo Mocker path exercises the request plumbing, not memory.
- **Eight job families, one machine, no humans.** H1b covers tools with parseable output; it says nothing about tools that report nothing until they finish or about human think time.

- **Curves keyed by tool name mislead when shapes mix.** Adding -j1 builds to training moved the -j2 error from 3.8 to 15.6. Command signatures (this build) address it but have not yet been re-scored on the collections.
- **Under-dispersion is bias, not variance.** Calibration cannot reach 90% coverage at short horizons; unseen sessions and tools need a fallback prior, not a wider one.
- **Fan-out is one level deep** in the forecaster and the simulator.
- **Hold cap of 600 s** in the reported runs is the core default, not the proxy's 60 s.

8. What is next

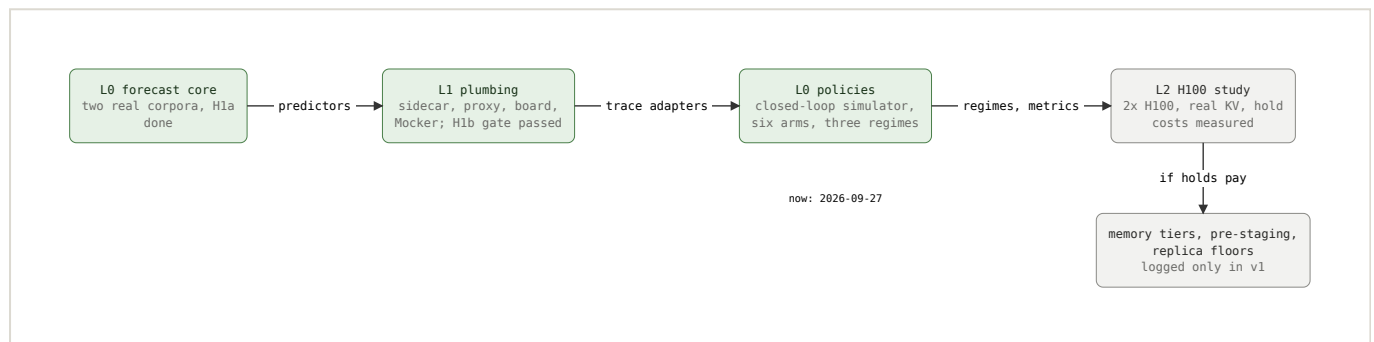


Figure 10. Demonstration ladder. Green is built and evidenced; grey is next. Explicit KV placement waits for a demonstrated mechanism (D3) and for the H100 study to show that holds pay.

8.1 Runs in flight at the time of writing

- **interactive_long_tool regime** (new): interactive sessions also run 120 s test suites with a strong signal and 300 s builds with a weak one, at loaded rates, hold cap 60 s. This is where M1 and M2 can separate in closed loop, because *when* interactive sessions return is now what the hold rule has to predict.
- **oracle_rule arm** (new): the same GDP-lite rule and index fed the true first-call demand per horizon read from the event heap. Whatever the forecast arms lose against it is forecast error; whatever it loses against native on background cost is the rule itself.
- **Loaded regime at a 60 s cap**, to see how much of the background cost was the long cap.

8.2 The two-GPU study (L2)

Two H100s running Dynamo with vLLM workers, the proxy in front, the same six arms plus the ablation and the same-rule oracle, mini-SWE-agent sessions on scripted Docker jobs as the background class and replayed interactive programs as the foreground. What must be recorded on the real engine is exactly the columns the simulator now logs: hold seconds, held KV block-seconds, evictions attributed to holds, recomputed prefill after resumption, holds at cap, and TTFT after tool return. The regimes start

from the loaded rates, not the base rates, because the base rates never stress a 2 s SLO. The study answers whether the simulator's ordering survives contact with a real KV cache; the design of the memory tiers waits on that answer.

8.3 Model work

- Re-score H1b with signature-keyed curves and an online per-session curve adaptation for suites seen once.
- A fallback prior for unseen tools, which is where the remaining coverage miss lives.
- Replay real trace tables (TraceLab, AgentX) through the simulator arms with prompt sizes from the corpus, as the calibration step before L2.

References

1. Project specification v1.1: `docs/superpowers/specs/2026-09-22-atfm-architecture-design.md` ; master plan `detail.md` .
2. Results notes: `docs/research/2026-09-22-h1-first-results.md` , `2026-09-23-h1-second-corpus-and-h1b.md` , `2026-09-23-l1-results.md` , `2026-09-24-h2sim-first-results.md` , `2026-09-27-dsec-paper-notes.md` .
3. DeepSeek, "DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale", arXiv 2609.22978, September 2026.
4. NVIDIA Dynamo v1.5: frontend, KV router, planner, Mocker workers; `nvext.agent_hints` request extension.
5. TraceLab agent session corpus; AgentX (SemiAnalysis) Claude Code session corpus.
6. ThunderAgent (agent-aware pause and resume of sessions), Continuum (KV time-to-live from tool start), ConServe (co-serving latency-sensitive and best-effort LLM work): the baselines this program compares against.

Diagrams authored in reladraw from the architecture sources; charts drawn to scale from the run outputs under `runs/` . Build script: `docs/paper/build_paper.py` .